A UNIFIED FRAMEWORK FOR AUTOMATED DISRUPTION LABELING OF DIII-D SHOTS USING A 1D CONVOLUTIONAL NEURAL NETWORK AND TEMPORAL FILTERING

Jason Jain, Dr. Jesse Rodriguez
Oregon State University
Coupled Systems Laboratory

July 23, 2026

ABSTRACT

Plasma in a Tokamak fusion reactor frequently experiences disruption events that result in a current drop to zero. The existence of a computationally inexpensive model for identifying disruptive shots and accurately identifying time of disruption can assist the scientific community by auto-labeling shots for further analysis. We took a labeled disruption dataset from the DIII-D Tokamak reactor, utilizing shot numbers 125500-186000 (nearly 40,000 total shots) to develop a new disruption labeling model that can generate automated labeling up to the current day. We train a CNN (Convolutional Neural Network) to perform boolean disruption/not-disruption prediction and achieve a 99% recall and 91% precision on our test dataset. We additionally develop a convolutional heuristic that accurately identifies time of disruption within a standard deviation of 2.1 µs.

1 Introduction

2 Neural Net Classifier

We were provided 39438 labeled shot files from the DIII-D team, of which 13429 were disruptive shots and labeled with time of disruption. Each file represented one shot and sampled reactor current at $40\,\mathrm{kHz}$, giving us a discrete current vector $\mathbf{I}$ where the mth component is current at the mth timestep.

$$\mathbf{I} = [I_1 \quad \dots \quad I_m] \tag{1}$$

We used an 80/10/10 train/test/dev split, which left us with 31542 trainable shots. To prepare the data for a neural net classifier, we tested three different nornmalization methods, including per-shot min-max unit scale normalization, per-shot Mean-Variance (meanvar) normalization, and dataset-wide meanvar normalization. We found most success with the dataset-wide meanvar approach. Letting the current mean and standard deviation across all shots be μ and σ, we computed a nornmalized shot tensor I_{N}.

$$\mathbf{I}_{\mathrm{N}} = \begin{cases} \frac{1}{\sigma}(\mathbf{I} - \mu) & \sigma > 0 \\ 0 & \sigma \leq 0 \end{cases} \tag{2}$$

The normalized data was then fed into a Convolutional Neural Network classifier (see **??**) consisting of four convolutional layers, each with a batch normalization layer and a Rectified Linear Unit (ReLU) activation function, and a final max pooling layer. The convolutional layer outputs were then fed into two fully connected layers, with batch normalization, ReLU, and a dropout layer to prevent overfitting, exposing a dropout rate `dropout` hyperparameter.

The CNN was trained using the Adam (Adaptive Moment Estimation) optimizer with weight decay and learning rate hyperparameters. Learning rate was tuned per-epoch with PyTorch's `lr_scheduler.ReduceLROnPlateau` in "min" mode, further exposing hyperparameters learning rate factor and learning rate patience. We utilized PyTorch's 'torch.nn.BCEWithLogitsLoss' with a tunable classification positive weight and gradient clipping. The final hyperparameters were number of training epochs, batch size, and early stopping patience.

We tuned our hyperparameters using Bayesian Optimization with the `ExpectedImprovement` acquisition function with $\xi = 0.05$, and we additionally inerted random hyperparameter samples every 5 trials to escape local minima. We used a weighted Fbeta=1.5 scoring strategy with a minimum precision floor of 90% and trained for 100+ trials on

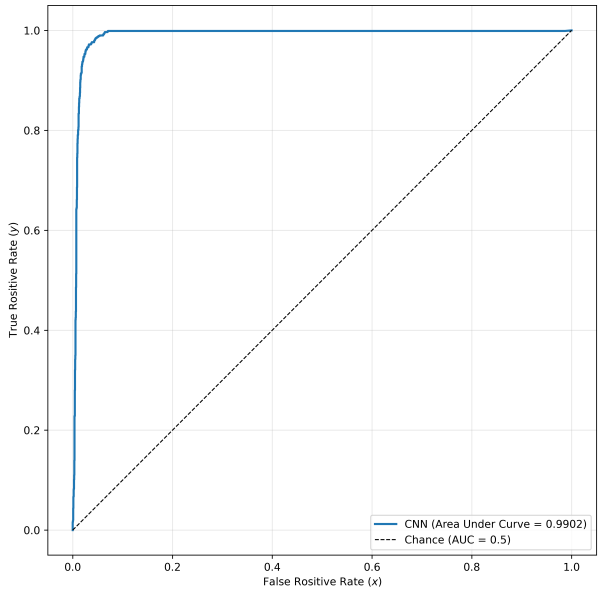

Figure 1: Figure caption.

the Polaris supercomputer in Argonne National Lab. Our final hyperparameters are presented in **??**.

Table 1: Tuned Hyperparameters

Hyperparameter	Value
Learning Rate	0.0005
Number of Epochs	200
Dropout Rate	0.35
Weight Decay	0.0001
Training Batch Size	128
Gradient Clip	1.0
Learning Rate Scheduler Factor	0.5
Learning Rate Patience	5
Early Stopping Patience	10
Positive Weight	3.0

The final performance metrics of the CNN classifier were an accuracy of a precision of 90.52% and recall of 98.84%, along with an F1 score of 0.9450 and F2 score of 0.9706.

3 Convolutional Heuristic for Time of Disruption

Given a noisy current $I(t)$ where $0 < t < T$, we first calculate a smoothed current by applying a moving average with a window size $K = T/S$, where $S = 300$ is an experimentally determined smoothing parameter.

$$I_{\text{smooth}}(t) = \frac{1}{K} \int_{t-K}^{t} I(\tau)\, d\tau \tag{3}$$

Note that if $\max(I) \approx 0$, we consider the shot a negative polarity shot and flip the current.

$$I_{\text{raw}}(t) = \begin{cases} -I(t) & \max(I) \approx 0 \\ I(t) & \text{otherwise} \end{cases} \tag{4}$$

We then define the time of disruption heuristic $f(t)$ as the difference between the raw $I(t)$ and smooth current (**??**). To find the start of a disruption, we search for the global maximum of $f(t)$, and then search for the following local minima to identify the disruption end.

We test this heuristic against the test holdout set, consisting of 889 total disruptive shots.

$$f(t) = I_{\text{raw}}(t) - I_{\text{smooth}}(t) \tag{5}$$

Computationally, the moving average is applied as a discrete convolution on the current tensor (**??**), where the ith component of the smooth current is $\frac{1}{K}\sum_{k=1}^{K} I_{i-k}$.

3.1 Subtleties

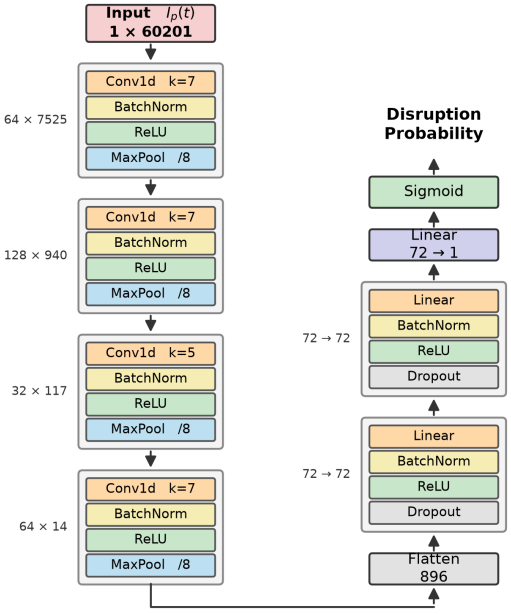

Figure 2: Figure caption.

4 Results

Report quantitative and qualitative findings. Reference figures and tables as needed.

5 Conclusion

Summarize findings and outline future work.

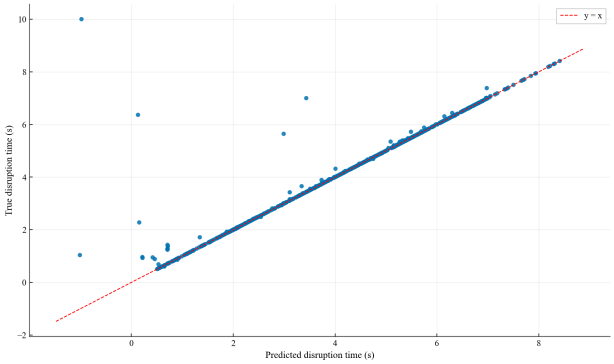

Figure 3: Figure caption.

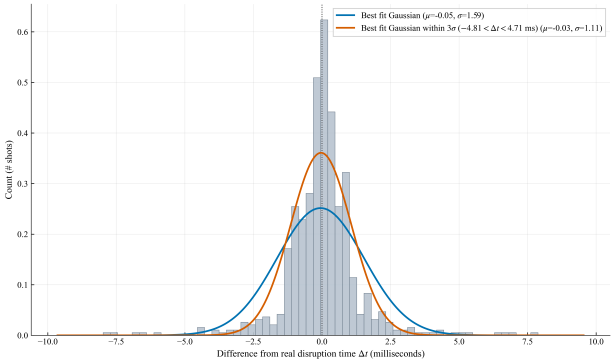

Figure 4: Figure caption.

Acknowledgments

Funding and thanks.

Data and code availability

Where artifacts are hosted.